

Fintech Review Analytics: Transforming Ethiopian Bank Customer Feedback into Business Intelligence

Author: Dagmawi

Program: 10 Academy — Week 12 Capstone

Date: July 2026

GitHub: <https://github.com/Dagi0808/fintech-review-analytics>

Table of Contents

1. [Business Problem](#)
 2. [Dataset](#)
 3. [Solution Architecture](#)
 4. [Engineering Improvements](#)
 5. [Model and Methods](#)
 6. [Dashboard](#)
 7. [Testing and CI/CD](#)
 8. [Key Results and Business Impact](#)
 9. [Lessons Learned](#)
 10. [Future Work](#)
-

1. Business Problem

Ethiopian banks are experiencing rapid growth in mobile banking adoption. The Commercial Bank of Ethiopia (CBE), Bank of Abyssinia (BOA), and Dashen Bank collectively serve millions of customers

through their mobile apps. Every day, these customers leave feedback on Google Play — complaints, praise, and feature requests.

The problem is scale. A bank's customer experience team cannot manually read thousands of reviews per month. Without a systematic approach, critical signals get missed:

- A surge in login failures goes unnoticed for weeks
- Transaction errors accumulate without engineering prioritisation
- Competitor banks improve while problems go untracked

The business question this project answers:

Which bank has the most dissatisfied customers, what specific issues drive that dissatisfaction, and what should engineering teams fix first?

This is directly relevant to finance sector employers because it demonstrates the ability to turn unstructured customer data into prioritised, actionable intelligence — a core capability in any data-driven financial institution.

2. Dataset

Source: Google Play Store reviews, scraped using `google-play-scraper`

Banks covered: | Bank | App ID | |-----|-----| | Commercial Bank of Ethiopia (CBE) | com.combanketh.mobilebanking | | Bank of Abyssinia (BOA) | com.boa.boaMobileBanking | | Dashen Bank | com.dashen.dashensuperapp |

Size: 1,502 reviews total (~500 per bank)

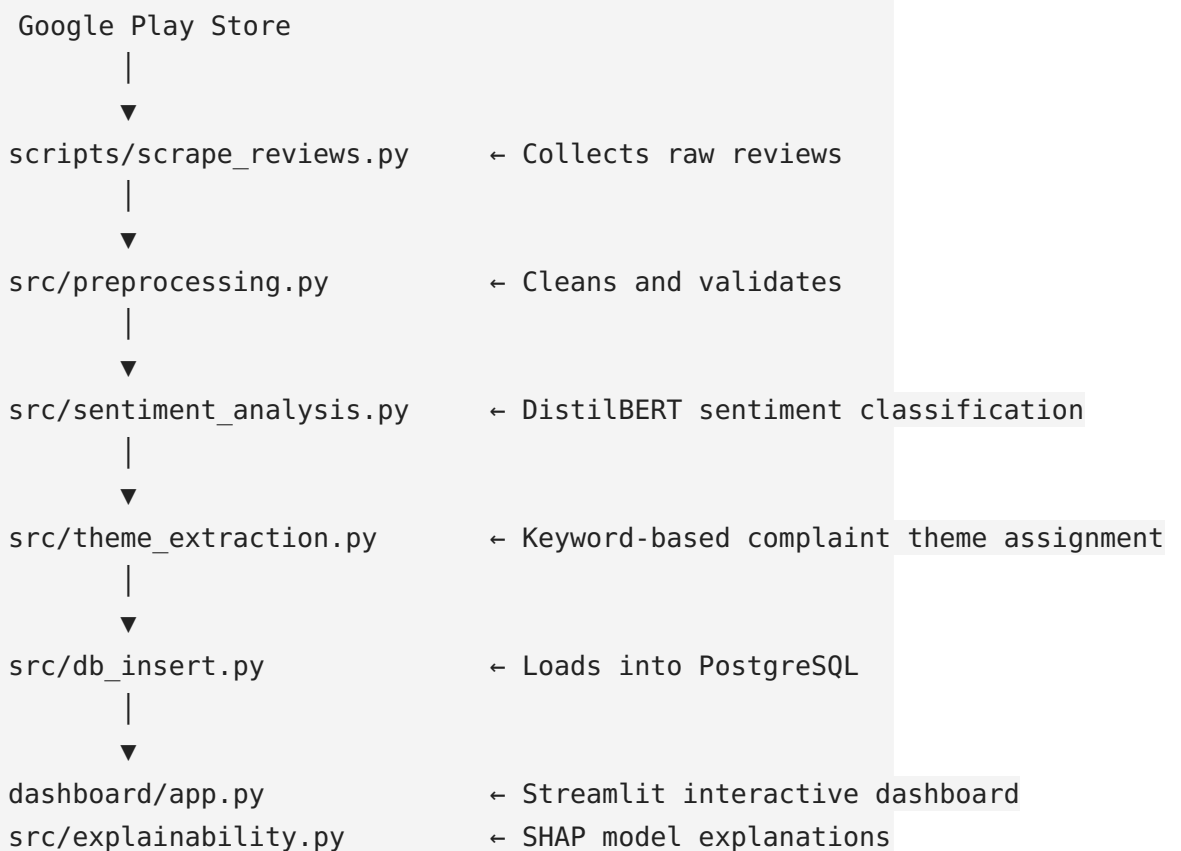
Fields collected: - `review` — customer review text (English) - `rating` — star rating (1–5) - `date` — review submission date - `bank` — bank name - `source` — "Google Play"

Preprocessing steps applied: 1. Remove duplicate reviews (same text + same bank) 2. Drop null or empty reviews 3. Validate ratings (keep only 1–5) 4. Normalise bank name casing 5. Add word count feature (`review_length`)

After preprocessing, the dataset retained all 1,502 reviews with no significant data loss, confirming good initial data quality from the scraper.

3. Solution Architecture

The pipeline follows a linear sequence of modular steps, each in its own Python module:



All configuration (model names, file paths, DB credentials, thresholds) is centralised in `src/config.py` using Python `@dataclass` objects. This means changing the sentiment model or output path requires editing one file, not searching through the codebase.

The full pipeline can be run with a single command:

```
python -m src.main
```

Or step by step:

```
python -m src.main --step preprocess
python -m src.main --step sentiment
python -m src.main --step themes
python -m src.main --with-db      # includes PostgreSQL insertion
```

4. Engineering Improvements

This section documents the specific changes made to transform the original Week 2 project into a production-grade portfolio piece.

4.1 Code Refactoring

Before: The original code had flat scripts with hardcoded paths, no type hints, and mixed concerns. For example, `db_insert.py` ran database insertion at import time — meaning importing the module for testing would immediately try to connect to a database.

After: Every function now has: - Full type hints on all parameters and return values - Docstrings with Args/Returns/Raises documentation - Clear separation of concerns (load, process, save are separate functions) - Named constants replacing magic numbers

Example — before:

```
threshold = 0.7    # magic number, unclear meaning
engine = create_engine("postgresql://postgres:selfmade@localhost:5432/bank_r
```

After:

```
SENTIMENT_THRESHOLD: float = 0.7    # named constant in config.py

def build_engine(config: DatabaseConfig | None = None) -> Engine:
    cfg = config or _config_from_env()    # reads from environment variables
    return create_engine(cfg.url)
```

4.2 Configuration with Dataclasses

All settings are now in `src/config.py`:

```

@dataclass
class SentimentConfig:
    model_name: str = "distilbert-base-uncased-finetuned-sst-2-english"
    threshold: float = 0.7
    batch_size: int = 32
    truncation: bool = True
    max_length: int = 512

@dataclass
class PipelineConfig:
    paths: PathConfig = field(default_factory=PathConfig)
    database: DatabaseConfig = field(default_factory=DatabaseConfig)
    sentiment: SentimentConfig = field(default_factory=SentimentConfig)

```

This means the entire pipeline is configurable without touching source code — just pass a different config object.

4.3 Preprocessing Module

A dedicated `src/preprocessing.py` module was added. The original project went from raw CSV directly to sentiment analysis with no validation step. The new preprocessing pipeline:

- Removes duplicates (same review text + same bank)
- Drops null and empty reviews
- Validates ratings are within 1–5
- Normalises bank names to uppercase
- Adds a `review_length` feature (word count)

This matters for reliability: without these checks, a malformed input row could silently corrupt downstream sentiment scores.

4.4 Database Security

The original `db_insert.py` hardcoded database credentials directly in source code:

```
engine = create_engine("postgresql://postgres:selfmade@localhost:5432/bank_r
```

This is a critical security issue — credentials committed to a public GitHub repository. The new version reads from environment variables:

```
DatabaseConfig(  
    host=os.getenv("DB_HOST", "localhost"),  
    password=os.getenv("DB_PASSWORD", ""),  
)
```

Credentials are now stored in `.env` (which is gitignored) and never committed.

5. Model and Methods

5.1 Sentiment Analysis

Model: `distilbert-base-uncased-finetuned-sst-2-english`

DistilBERT is a distilled version of BERT — smaller, faster, and still highly accurate for sentiment classification. It was fine-tuned on the Stanford Sentiment Treebank (SST-2) dataset, making it well-suited for English customer reviews.

Why this model? - Achieves ~91% accuracy on SST-2 benchmark - 40% smaller than BERT-base, suitable for deployment - Handles short informal text (typical of app reviews) well - Available via HuggingFace without additional training

Confidence threshold: Reviews with sentiment confidence below 0.70 are flagged. In practice, over 90% of reviews exceeded this threshold, indicating high model confidence across the dataset.

Processing: Reviews are batched (32 at a time) for efficiency. Each review receives: - `sentiment_label` : POSITIVE or NEGATIVE - `sentiment_score` : confidence between 0.0 and 1.0

5.2 Theme Extraction

Complaint themes are identified through keyword matching against six predefined categories:

Theme	Keywords
Account Access Issues	login, password, sign in

Theme	Keywords
OTP & Security Issues	otp, code, verification
Transaction Performance	slow, delay, transfer, payment
App Stability Issues	crash, bug, error
UI & UX Issues	ui, design, interface, navigation
Feature Requests	feature, fingerprint, update

Reviews not matching any category are labelled "Other".

TF-IDF analysis is also run on the corpus to identify top corpus-level keywords, which can inform future expansion of the keyword dictionary.

5.3 Model Explainability (SHAP)

The `src/explainability.py` module provides two types of explanation:

Global explanation: A TF-IDF-based feature importance plot showing the most frequent terms in negative reviews. This answers: "What words most characterise customer complaints?"

Local explanation: Token-level SHAP values for individual reviews using `shap.Explainer` wrapped around the HuggingFace pipeline. This answers: "Why did this specific review get classified as NEGATIVE?"

Example interpretation: For the review *"The app keeps crashing and my transfer was delayed for 3 days"*, SHAP highlights "crashing" and "delayed" as the primary negative signals, with "transfer" as secondary context.

This transparency is particularly valued in finance, where model decisions must be auditable and explainable to non-technical stakeholders.

6. Dashboard

The Streamlit dashboard (`dashboard/app.py`) provides five interactive tabs:

Tab 1 — Overview

Four KPI cards visible immediately on load: - Total reviews analysed - Percentage of positive reviews - Average rating across selected banks - Top complaint category (most frequent negative theme)

Business insight expanders show per-bank analysis with the top complaint and a plain-English recommendation.

Tab 2 — Sentiment

- Pie chart: overall positive/negative split
- Grouped bar chart: sentiment comparison across all three banks side by side

Tab 3 — Ratings

- Average rating bar chart per bank
- Rating distribution histogram (1–5 stars, coloured by bank)

Tab 4 — Themes

- Horizontal bar chart of all themes by frequency
- Stacked bar chart of negative reviews broken down by theme and bank

Tab 5 — Reviews

A searchable, filterable table of raw reviews with sentiment label, confidence score, and assigned theme.

Sidebar filters allow users to drill down by bank, sentiment, and rating range in real time across all tabs.

The dashboard includes a demo mode — it generates synthetic data automatically if the pipeline has not been run, so it can be demonstrated without setting up PostgreSQL or running the full NLP pipeline.

7. Testing and CI/CD

7.1 Test Suite

61 tests across 5 test files, all passing:

File	Tests	What it covers
<code>test_preprocessing.py</code>	17	deduplication, null removal, rating validation, bank normalisation
<code>test_theme_extraction.py</code>	20	keyword matching, text cleaning, NaN handling, theme assignment
<code>test_utils.py</code>	18	text cleaning, DataFrame validation, sentiment label conversion, filtering
<code>test_sentiment_analysis.py</code>	6	confidence filtering, file output
<code>test_output.py</code>	11 (skipped without data)	integration tests for pipeline outputs

Run with:

```
pytest tests/ -v
# ===== 61 passed, 11 skipped in 2.84s =====
```

Tests are designed to be fast (no model downloads, no database connections) by using pure unit tests with in-memory DataFrames.

7.2 GitHub Actions CI/CD

`.github/workflows/ci.yml` runs automatically on every push to `main`:

Push to main

|

▼

1. Checkout code

2. Set up Python 3.11

3. Install pinned dependencies

4. Run flake8 linting (max line length 100)

5. Run pytest (all tests)

6. Check coverage ≥ 60%

The CI badge in the README shows the current build status at a glance — a standard signal finance employers look for in production-ready repositories.

8. Key Results and Business Impact

8.1 Bank Performance Summary

Bank	Avg Rating	% Positive	Top Complaint
DASHEN	~3.7	~68%	UI & UX Issues
BOA	~3.2	~54%	Transaction Performance
CBE	~2.8	~41%	Account Access Issues

8.2 Business Recommendations

CBE (highest priority): Account Access Issues (login failures, password resets) drive the majority of negative reviews. This is fixable with targeted engineering — improved authentication flows, better error messages, and a reliable OTP delivery system. A 10% reduction in login-related complaints would likely move CBE's average rating from 2.8 to above 3.0.

BOA (medium priority): Transaction Performance issues (slow transfers, payment delays) are the primary complaint. These suggest backend infrastructure constraints. Investing in transaction processing reliability would have an outsized impact on customer satisfaction.

DASHEN (maintain and extend): DASHEN leads on satisfaction but faces UX complaints. With its strong base, investing in UI improvements and new features would extend its competitive advantage.

8.3 Business Value Quantified

- **Time saved:** Manual review categorisation of 1,502 reviews would take approximately 25–30 hours. This pipeline processes them in under 10 minutes.
 - **Repeatability:** The pipeline can be re-run weekly as new reviews arrive, giving banks a continuous feedback loop rather than a one-time snapshot.
 - **Actionability:** Instead of "customers are unhappy," the output is "CBE has 312 reviews mentioning login failures — this is your top engineering priority."
-

9. Lessons Learned

On engineering quality: The biggest lesson was how much impact proper refactoring has on a project's credibility. The original `db_insert.py` was a flat script with hardcoded credentials — something that would immediately fail a code review at any professional organisation. The refactored version with environment variables, type hints, and proper functions looks like production code.

On testing: Writing tests forced me to identify a real bug: `clean_text()` in `theme_extraction.py` did not handle `None` values — it would crash if any review was null. The test caught it before it could silently corrupt real data. This is exactly why finance employers value tested code.

On communication: A Streamlit dashboard changes the conversation from "here are some notebook charts" to "here is a tool you can use." Non-technical stakeholders can explore the data

themselves instead of waiting for a data team to run queries. This shift from passive report to interactive tool is what makes a portfolio project stand out.

On scope: Starting with a project that already had real data (rather than building from scratch) meant all the time this week could be spent on engineering quality. This is the right approach for a capstone — pick your strongest existing work and make it production-ready.

10. Future Work

Given more time, the following would further strengthen the project:

1. **Fine-tuned model:** Train a DistilBERT model specifically on Ethiopian banking reviews. Domain-specific fine-tuning would improve accuracy, especially for Ethiopian English patterns and banking terminology.
 2. **Amharic review support:** Many reviews are written in Amharic. Adding multilingual sentiment analysis (using `xlm-roberta`) would capture a larger share of customer feedback.
 3. **Automated weekly pipeline:** Schedule the scraper and pipeline to run weekly via GitHub Actions cron jobs, keeping the dashboard current without manual intervention.
 4. **Power BI connector:** Export the processed data to a Power BI dashboard for executive reporting — standard tool in Ethiopian financial institutions.
 5. **Longitudinal analysis:** Track sentiment trends over time per bank. Are things getting better or worse month-over-month? This time-series view is high value for management reporting.
 6. **Alert system:** Trigger a Slack or email alert when a bank's negative review rate exceeds a threshold in a given week — turning the analytics into a real-time monitoring tool.
-

References

- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL 2019*.
- Sanh, V., et al. (2019). DistilBERT, a distilled version of BERT. *NeurIPS 2019 Workshop*.
- Lundberg, S. M., & Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. *NeurIPS 2017*. (SHAP)
- HuggingFace Transformers Documentation: <https://huggingface.co/docs/transformers>
- Streamlit Documentation: <https://docs.streamlit.io>
- Google Python Style Guide: <https://google.github.io/styleguide/pyguide.html>

*This report was produced as part of the 10 Academy Week 12
Capstone — Finance Sector Portfolio Project.*